

1.

```
#include <stdio.h>

struct a{
    int matricula;
    char nome[100];
    float prova1, prova2, prova3;
} aluno;

int main(){

    printf("Tamanho da struct: %d", sizeof(struct a));

    return 0;
}
```

2.

```
#include <stdio.h>
#include <stdlib.h>

typedef struct{
    char nome[100];
    int idade;
    char endereco[100];
} pessoa;

void pp(int N, pessoa *p){
    p = (pessoa*) malloc(N * sizeof(pessoa));
    for(int i = 0; i < N; i++){
        printf("Coloque o nome: ");
        scanf("%s", p[i].nome);
        printf("Coloque a idade: ");
        scanf("%d", &p[i].idade); //& pq eh do tipo int, as outras
variaveis como sao strings nao eh necessario
        printf("Coloque o endereco: ");
        scanf("%s", p[i].endereco);
    }
    free(p);
}

int main(){

    pessoa *p;
```

```

int N;

printf("Quantas pessoas serao registradas: ");
scanf("%d", &N);

p = (pessoa *)malloc(N * sizeof(pessoa));
pp(N, p);

return 0;
}

```

3.

```

#include <stdio.h>
#include <stdlib.h>

typedef struct{
    int variavel[10000];
} vet;

vet* vetor(int tam){
    vet *v = (vet*) malloc(tam * sizeof(vet));

    for(int i = 0; i < tam; i++){
        printf("Coloque um numero: ");
        scanf("%d", &v[i].variavel[i]);
    }

    printf("O vetor colocado: \n");
    for(int i = 0; i < tam; i++){
        printf("%d ", v[i].variavel[i]);
    }

    return v;
}

int main() {
    int tam;

    printf("Coloque um tamanho de um vetor: ");
    scanf("%d", &tam);

    vet *v = vetor(tam);
}

```

```
    free(v);

    return 0;
}
```

4.

```
#include <stdio.h>
#include <stdlib.h>

int** matriz(int linhas, int colunas) {
    int **mat = (int **)malloc(linhas * sizeof(int *));
    for(int i = 0; i < linhas; i++){
        mat[i] = (int *)malloc(colunas * sizeof(int));
        for(int j = 0; j < colunas; j++){
            printf("Coloque um numero na linha %d e na coluna %d da
matriz: ", i, j);
            scanf("%d", &mat[i][j]);
        }
    }
    return mat;
}

void liberarMatriz(int **mat, int linhas) {
    for(int i = 0; i < linhas; i++){
        free(mat[i]);
    }
    free(mat);
}

int numero(int **mat, int linhas, int colunas, int num){
    for(int i = 0; i < linhas; i++){
        for(int j = 0; j < colunas; j++){
            if(num == mat[i][j]){
                return 1;
            }
        }
    }
    return 0;
}

int main(){
    int linhas, colunas;
```

```

printf("Quantas linhas tera sua matriz: ");
scanf("%d", &linhas);
printf("Quantas colunas tera na sua matriz: ");
scanf("%d", &colunas);

int **mat = matriz(linhas, colunas);

int num;
printf("Coloque um numero para ver se esta na matriz colocada: ");
scanf("%d", &num);

if(numero(mat, linhas, colunas, num)){
    printf("O numero %d está presente na matriz.\n", num);
}else{
    printf("O numero %d não está presente na matriz.\n", num);
}

liberarMatriz(mat, linhas);

return 0;
}

```

5.

```

#include <stdio.h>
#include <stdlib.h>

int* numero(int N){
    if(N <= 0){
        return NULL;
    }

    int *vetor = (int*) malloc(N * sizeof(int));
    return vetor;
}

int main(){

    int N;

    printf("Coloque um numero inteiro positivo: ");
    scanf("%d", &N);
}

```

```

int *vetor = numero(N);

if(vetor == NULL){
    printf("Ponteiro nulo");
}else{
    printf("Vetor alocado!");
}

free(vetor);

return 0;
}

```

6.

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

char *inverte(char *string){
    size_t tamanho = strlen(string);

    char *invertido = (char*)malloc((tamanho + 1) * sizeof(char)); //+1
    para o \0

    for(size_t i = 0; i < tamanho; i++){
        invertido[i] = string[tamanho - i - 1];
    }

    return invertido;
}

int main(){

    char string[100000];

    printf("Coloque uma palavra ou frase: ");
    fgets(string, sizeof(string), stdin);

    char *stringinvertida = inverte(string);

    printf("A palavra invertida: %s", stringinvertida);
}

```

```
    free(stringinvertida);

    return 0;
}
```

7.

```
#include <stdio.h>
#include <stdlib.h>

int *novo(int N, int valor){
    if(N <= 0){
        return NULL;
    }

    int *vetor = (int*)malloc(N * sizeof(int));

    for(int i = 0; i < N; i++){
        vetor[i] = valor;
    }

    return vetor;
}

int main(){

    int N, valor;

    printf("Coloque um numero inteiro positivo para o tamanho do vetor: ");
    scanf("%d", &N);

    printf("Coloque um valor para preenches o vetor: ");
    scanf("%d", &valor);

    int *mvet = novo(N, valor);

    if(mvet == NULL){
        printf("Numero invalido");
    }else{
        printf("Vetor criado: \n");
        for(int i = 0; i < N; i++){
            printf("%d ", mvet[i]);
        }
    }
}
```

```

    }

}

free(mvet);

return 0;
}

```

8.

```

#include <stdio.h>
#include <stdlib.h>

int* somaColunas(int **A, int N){
    int *B = (int*)malloc(N * sizeof(int));
    for(int i = 0; i < N; i++){
        B[i] = 0;
        for(int j = 0; j < N; j++){
            B[i] += A[j][i];
        }
    }
    return B;
}

int main(){

    int N;

    printf("Coloque um numero para representar o numero de linhas e
colunas: ");
    scanf("%d", &N);

    int **A = (int**)malloc(N * sizeof(int)); //alocar memoria antes de
colocar os valores
    for(int i = 0; i < N; i++){
        A[i] = (int*)malloc(N * sizeof(int));
    }

    for(int i = 0; i < N; i++){
        for(int j = 0; j < N; j++){
            printf("Coloque um numero: \n");
            scanf("%d", &A[i][j]);
        }
    }
}

```

```

    int *B = somaColunas(A, N);

    printf("Os resultados sao: \n");
    for(int i = 0; i < N; i++){
        printf("%d ", B[i]);
    }

    free(A);
    free(B);

    printf("\n");

    return 0;
}

```

9.

```

#include <stdio.h>
#include <stdlib.h>

int** multpmatriz(int **A, int linhaA, int colunaA, int **B, int
linhaB, int colunaB){
    if(colunaA != linhaB){
        printf("Multiplicação de matrizes nao eh possivel.\n");
        return NULL;
    }

    int **C = (int**)malloc(linhaA * sizeof(int));
    for(int i = 0; i < linhaA; i++){
        C[i] = (int*)malloc(colunaB * sizeof(int));
    }

    for(int i = 0; i < linhaA; i++){
        for(int j = 0; j < colunaB; j++){
            C[i][j] = 0;
            for(int k = 0; k < colunaA; k++){
                C[i][j] += A[i][k] * B[k][j];
            }
        }
    }

    free(C);
}

```



```

        return C;
    }

int main(){

    int linhaA, colunaA, linhaB, colunaB;

    printf("Coloque a quantidade de linhas e colunas da matriz A: ");
    scanf("%d%d", &linhaA, &colunaA);
    printf("Coloque a quantidade de linhas e colunas da matriz B: ");
    scanf("%d%d", &linhaB, &colunaB);

    int **matriz1 = (int**)malloc(linhaA * sizeof(int));
    int **matriz2 = (int**)malloc(linhaB * sizeof(int));

    for(int i = 0; i < linhaA; i++){
        matriz1[i] = (int*)malloc(colunaA * sizeof(int));
    }

    for(int i = 0; i < linhaB; i++){
        matriz2[i] = (int*)malloc(colunaB * sizeof(int));
    }

    printf("Coloque os valores da matriz A: ");
    for(int i = 0; i < linhaA; i++){
        for(int j = 0; j < colunaA; j++){
            printf("Coloque o valor para a linha %d e para a coluna %d", i, j);
        }
    }

    printf("Coloque os valores da matriz B: ");
    for(int i = 0; i < linhaB; i++){
        for(int j = 0; j < colunaB; j++){
            printf("Coloque o valor para a linha %d e para a coluna %d", i, j);
        }
    }

    int **matrizresultado = multpmatriz(matriz1, linhaA, colunaA,
matriz2, linhaB, colunaB);

    if(matrizresultado == NULL){

```

```

        printf("Não foi possível realizar a multiplicação das
matrizes.\n");
    }
    else{
        for(int i = 0; i < linhaA; i++){
            for(int j = 0; j < colunaA; j++){
                printf("%d", matrizresultado[i][j]);
            }
        }

        free(matrizresultado);

        return 0;
    }
}

```

10.

```

#include <stdio.h>
#include <stdlib.h>

typedef struct{
    int codigo;
    char nome[100];
    int quantidade;
    float preco;
} produto;

produto* product(int n){
    produto *p = (produto*) malloc(n * sizeof(produto));
    for(int i = 0; i < n; i++){
        printf("Codigo: ");
        scanf("%d", &p[i].codigo);
        printf("Nome: ");
        scanf("%s", p[i].nome);
        printf("Quantidade: ");
        scanf("%d", &p[i].quantidade);
        printf("Preco: ");
        scanf("%f", &p[i].preco);
    }
    return p;
}

void pmaiorquantidade(produto *p, int n){

```

```

int maiorquantidade = 0;
int indice = -1;

for(int i = 0; i < n; i++){
    if(p[i].quantidade > maiorquantidade){
        maiorquantidade = p[i].quantidade;
        indice = i;
    }
}

if(indice != -1){
    printf("Produto com maior quantidade disponivel: \n");
    printf("Codigo: %d\n", p[indice].codigo);
    printf("Nome: %s\n", p[indice].nome);
    printf("Quantidade: %d\n", p[indice].quantidade);
    printf("Preco: %.2f\n", p[indice].preco);
}
else{
    printf("Nao ha produtos com quantidade disponivel.\n");
}
}

int main(){
    int n;
    printf("Coloque quantos produtos: ");
    scanf("%d", &n);

    produto *p = product(n);

    pmaiorquantidade(p, n);

    free(p);

    return 0;
}

```

11.

```

#include <stdio.h>
#include <stdlib.h>

int comparanum(int *lot, int *mlot){
    int count = 0, i, j;

```

```

        for(i = 0; i < 6; i++){
            for(j = 0; j < 6; j++){
                if(lot[i] == mlot[j]){
                    count++;
                    break;
                }
            }
        }

        return count;
    }

void numcerto(int *lot, int *mlot, int count){
    int *corretos = (int*) malloc(count * sizeof(int));
    if(corretos == NULL){
        printf("Erro ao alocar memoria\n");
        exit(1);
    }

    count = 0;
    for(int i = 0; i < 6; i++){
        for(int j = 0; j < 6; j++){
            if(lot[i] == mlot[j]){
                corretos[count++] = lot[i];
                break;
            }
        }
    }

    printf("Seus numeros corretos: ");
    for(int i = 0; i < count; i++){
        printf("%d ", corretos[i]);
    }

    printf("\n");

    free(corretos);
}

int main(){
    int lot[6], mlot[6], i;

```

```

    printf("Coloque os 6 primeiros numeros gerados pela loteria em
ordem. ");
    for(i = 0; i < 6; i++){
        printf("Coloque o numero: ");
        scanf("%d", &lot[i]);
    }

    printf("Digite seus 6 numeros em ordem. ");
    for(i = 0; i < 6; i++){
        printf("Coloque o numero: ");
        scanf("%d", &mlot[i]);
    }

    int count = comparanum(lot, mlot);

    printf("Quantidade de numeros corretos: %d\n", count);

    printf("Numeros sorteados: ");
    for(i = 0; i < 6; i++){
        printf("%d ", lot[i]);
    }

    printf("\n");

    numcerto(lot, mlot, count);

    return 0;
}

```

12.

```

#include <stdio.h>
#include <stdlib.h>

void colocardado(int *mem, int memoria, int endereco, int valor){
    if(endereco >= 0 && endereco < memoria){
        mem[endereco] = valor;
        printf("Dado inserido com sucesso.\n");
    }else{
        printf("Endereco invalido.\n");
    }
}

void verdado(int *mem, int memoria, int endereco){

```

```

        if(endereco >= 0 && endereco < memoria){
            printf("O valor no endereco %d eh: %d\n", endereco,
mem[endereco]);
        }else{
            printf("Endereco invalido.\n");
        }
    }
}

int main(){

    int memoria, endereco, valor;
    char opcao;

    printf("Quantidade de memoria do 'computador': ");
    scanf("%d", &memoria);

    int *mem = (int*)malloc(memoria * sizeof(int));
    if(mem == NULL){
        printf("Erro: memoria insuficiente.\n");
        return 1;
    }

    for(int i = 0; i < memoria; i++){
        mem[i] = 0;
    }

    printf("Deseja inserir (i) ou consultar (c) um dado? ");
    scanf(" %c", &opcao);

    if(opcao == 'i'){
        printf("Informe o endereco (0-%d): ", memoria-1);
        scanf("%d", &endereco);

        printf("Informe o valor a ser inserido: ");
        scanf("%d", &valor);

        colocardado(mem, memoria, endereco, valor);
    }else if(opcao == 'c'){
        printf("Informe o endereco (0-%d): ", memoria-1);
        scanf("%d", &endereco);

        verdado(mem, memoria, endereco);
    }else{

```

```
        printf("Opcao invalida.\n");  
    }  
  
    free(mem);  
  
    return 0;  
}
```